

Plantilla SDD – Proyecto Graficador Interactivo en Python

1. Información Plantilla SDD – Proyecto Graficador Interactivo en Python

1. Información General del Proyecto

Nombre del Proyecto:
Graficador Interactivo 2D en Python

Versión:
1.0

Fecha:
11 de mayo de 2026

Autor(es):
[Pendiente por definir]

Stakeholders:

- **Estudiantes del curso de gráficos computacionales**
- **Docente evaluador**
- **Desarrolladores del proyecto**

Descripción General:

El sistema consiste en una aplicación gráfica interactiva desarrollada en Python utilizando la biblioteca Pygame. El software permitirá la representación visual de algoritmos clásicos de

rasterización y generación de figuras geométricas en 2D mediante implementación manual de algoritmos gráficos.

Objetivo del Sistema:

Desarrollar un graficador interactivo que permita visualizar y manipular figuras geométricas usando algoritmos gráficos implementados manualmente, promoviendo el aprendizaje de rasterización y modularidad de software.

Alcance:

El sistema permitirá:

- **Dibujar líneas mediante DDA y Bresenham**
- **Dibujar circunferencias mediante Bresenham**
- **Dibujar curvas Bézier cúbicas**
- **Dibujar triángulos, rectángulos, polígonos y elipses**
- **Seleccionar figuras desde una interfaz gráfica**
- **Configurar colores de dibujo**
- **Limpiar y reiniciar el lienzo**
- **Mantener separación entre lógica gráfica e interfaz**

No se incluirán:

- **Renderizado 3D**
 - **Persistencia en bases de datos**
 - **Exportación de imágenes**
 - **Uso de funciones gráficas predefinidas como `pygame.draw.line()`**
-

2. Visión del Sistema

Problema que resuelve

Permite comprender visualmente el funcionamiento interno de algoritmos gráficos clásicos mediante implementación manual y visualización interactiva.

Usuarios objetivo

- **Estudiantes de programación gráfica**
- **Docentes de computación gráfica**
- **Desarrolladores en formación**

Beneficios esperados

- **Comprensión práctica de algoritmos de rasterización**
- **Aprendizaje de modularización en Python**
- **Visualización inmediata de figuras geométricas**
- **Aplicación de buenas prácticas de programación**

Contexto de uso

Aplicación educativa ejecutada en computadores con Python y Pygame instalados.

3. Requisitos Funcionales (RF)

ID	Requisito	Descripción	Prioridad	Actor
RF-01	Ventana principal	Mostrar área de dibujo y panel de control	Alta	Usuario
RF-02	Selección de figuras	Permitir seleccionar figuras geométricas	Alta	Usuario
RF-03	Algoritmo DDA	Dibujar líneas usando DDA	Alta	Usuario
RF-04	Algoritmo Bresenham línea	Dibujar líneas usando Bresenham	Alta	Usuario
RF-05	Circunferencia Bresenham	Dibujar circunferencias manualmente	Alta	Usuario
RF-06	Curvas Bézier	Dibujar curvas Bézier cúbicas	Alta	Usuario

RF -07	Formas cerradas	Dibujar triángulos, rectángulos y polígonos	Medi a	Usuari o
RF -08	Dibujar elipses	Generar elipses en el lienzo	Medi a	Usuari o
RF -09	Configuració n de color	Seleccionar color de dibujo	Medi a	Usuari o
RF -10	Limpiar lienzo	Reiniciar el área de dibujo	Alta	Usuari o
RF -11	Renderizado inmediato	Visualizar figuras inmediatamente	Alta	Usuari o
RF -12	Modularidad	Separar lógica gráfica e interfaz	Alta	Sistem a
RF -13	Gestión de eventos	Capturar teclado y mouse	Alta	Sistem a
RF -14	Documentaci ón técnica	Incluir comentarios y documentación PEP8	Alta	Desarr ollador

4. Requisitos No Funcionales (RNF)

ID	Tipo	Descripción
RNF-01	Rendimiento	Las figuras deben renderizarse en tiempo real
RNF-02	Modularidad	Separación entre interfaz y algoritmos
RNF-03	Mantenibilidad	Código organizado por módulos
RNF-04	Estándar	Código documentado siguiendo PEP8
RNF-05	Compatibilidad	Compatible con Python y Pygame
RNF-06	Usabilidad	Interfaz clara e intuitiva
RNF-07	Restricción técnica	No usar primitivas gráficas predefinidas de Pygame

RNF -08	Escalabilidad	Posibilidad de agregar nuevas figuras
--------------------	----------------------	--

RNF -09	Legibilidad	Funciones pequeñas y reutilizables
--------------------	--------------------	---

RNF -10	Calidad visual	Las figuras deben visualizarse correctamente
--------------------	-----------------------	---

5. Casos de Uso

5.1 Diagrama de Casos de Uso

- **Seleccionar figura**
 - **Seleccionar color**
 - **Dibujar línea DDA**
 - **Dibujar línea Bresenham**
 - **Dibujar circunferencia**
 - **Dibujar curva Bézier**
 - **Dibujar polígonos**
 - **Dibujar elipse**
 - **Limpiar lienzo**
 - **Reiniciar aplicación**
-

5.2 Especificación de Casos de Uso

Nombre:

Dibujar Línea DDA

Actor:

Usuario

Descripción:

Permite dibujar una línea usando el algoritmo DDA.

Precondiciones:

- **La aplicación debe estar iniciada**
- **El usuario debe seleccionar la herramienta DDA**

Flujo principal:

- 1. El usuario selecciona DDA**
- 2. El usuario define punto inicial**
- 3. El usuario define punto final**
- 4. El sistema calcula los píxeles**
- 5. El sistema renderiza la línea**

Flujos alternativos:

- **Cancelación de selección de puntos**

Postcondiciones:

- **Línea renderizada en el lienzo**

6. Modelo de Datos

6.1 Diccionario de Datos

Entidad	Atributo	Tipo	Descripción
Figura	tipo	String	Tipo de figura
Figura	color	Tuple	Color RGB
Punto	x	Integer	Coordenada horizontal
Punto	y	Integer	Coordenada vertical
CurvaBezier	puntos_control	List	Puntos de control
Interfaz	herramienta_activa	String	Figura seleccionada

6.2 Diagrama Entidad-Relación (DER)

- **Figura contiene múltiples puntos**
- **CurvaBezier utiliza 4 puntos de control**

- Interfaz controla herramienta y color actual
-

7. Arquitectura del Sistema

7.1 Estilo Arquitectónico

Arquitectura modular por capas.

7.2 Diagrama de Componentes

- `main.py`
 - `ui/`
 - `algorithms/`
 - `models/`
 - `utils/`
-

7.3 Diagrama de Paquetes

`project/`

|

|— `main.py`

|— `ui/`

| |— `interface.py`

- | └─ controls.py
- |
- | └─ algorithms/
 - | └─ dda.py
 - | └─ bresenham.py
 - | └─ circle.py
 - | └─ bezier.py
 - | └─ ellipse.py
- |
- | └─ models/
 - | └─ point.py
 - | └─ figure.py
- |
- | └─ utils/
 - | └─ constants.py
 - | └─ helpers.py

7.4 Diagrama de Clases

Clase: Point

- **x**
- **y**

Clase: Figure

- **color**
- **draw()**

Clase: DDAAAlgorithm

- **calculate_points()**

Clase: BresenhamAlgorithm

- **calculate_points()**

Clase: InterfaceManager

- **handle_events()**
 - **render()**
-

8. Contratos de Interfaz (UI/UX)

Pantalla	Evento	Acción	Destino
-----------------	---------------	---------------	----------------

Menú principal	Seleccionar figura	Activar herramienta	Lienzo
Lienzo	Click mouse	Capturar puntos	Algoritmo
Panel lateral	Seleccionar color	Cambiar color actual	Lienzo
Panel lateral	Limpiar	Reiniciar lienzo	Lienzo

9. Reglas de Negocio

ID	Regla	Descripción
RN-01	Implementación manual	Todos los algoritmos deben implementarse manualmente
RN-02	Restricción gráfica	No usar <code>pygame.draw.line()</code>

RN- 03 Curva Bézier **Debe utilizar 4 puntos de control**

RN- 04 Modularidad **Separar GUI y lógica**

RN- 05 Color activo **Todas las figuras usan el color seleccionado**

RN- 06 Interacción **La figura debe mostrarse inmediatamente**

RN- 07 Documentación **Todas las funciones deben documentarse**

10. Especificación de Algoritmos

Nombre del algoritmo:

DDA

Entrada:

- **Punto inicial (x1, y1)**
- **Punto final (x2, y2)**

Salida:

Lista de píxeles de la línea.

Descripción:

Calcula incrementos en X y Y para interpolar los puntos de una línea.

Inicio

- 1. Calcular dx y dy**
- 2. Determinar pasos**
- 3. Calcular incrementos**
- 4. Iterar sobre puntos**
- 5. Dibujar píxeles**

Fin

11. API / Servicios

Endp oint	Méto do	Descripción
No aplica	No aplica a	Aplicación local sin servicios externos

12. Trazabilidad

RF	Caso de Uso	Módulo	Prue ba
RF- 03	Dibujar línea DDA	dda.py	PT-0 1
RF- 04	Línea Bresenham	bresenh am.py	PT-0 2
RF- 05	Circunfere ncia	circle.p y	PT-0 3
RF- 06	Curva Bézier	bezier.p y	PT-0 4
RF- 10	Limpiar lienzo	interfac e.py	PT-0 5

13. Plan de Pruebas

ID	Descripc ión	Entrada	Resultado Esperado
-----------	-------------------------	----------------	-------------------------------

PT- 01	Línea DDA	Dos puntos	Línea correcta
PT- 02	Bresenh am	Dos puntos	Línea rasterizada
PT- 03	Circunfe rencia	Centro y radio	Circunferenc ia visible
PT- 04	Bézier	4 puntos	Curva renderizada
PT- 05	Limpiar lienzo	Botón limpiar	Lienzo vacío

14. Criterios de Aceptación

El sistema debe permitir:

- **Seleccionar figuras desde la interfaz**
- **Dibujar figuras en tiempo real**
- **Cambiar colores**
- **Limpiar el lienzo**
- **Visualizar algoritmos manuales**

El sistema debe validar:

- **Entrada correcta de puntos**
 - **Selección válida de herramientas**
 - **Separación modular del código**
 - **Cumplimiento de documentación PEP8**
-

15. Gestión del Proyecto

Metodología:

Desarrollo incremental por módulos.

Herramientas:

- **Python**
- **Pygame**
- **VS Code**
- **Git**

Cronograma:

- **Diseño de interfaz**
 - **Implementación de algoritmos**
 - **Integración GUI**
 - **Pruebas**
 - **Documentación**
 - **Video demostrativo**
-

16. Integración con Agentes de IA

16.1 Roles

Agente	Responsabilidad
Agente Arquitecto	Diseñar arquitectura modular
Agente Desarrollador	Implementar algoritmos
Agente QA	Validar funcionamiento
Agente Documentador	Generar documentación técnica

16.2 Prompt Base

Actúa como un desarrollador experto en Python y Pygame.

Usa esta especificación SDD para implementar un graficador interactivo 2D.

Restricciones:

- No inventar requisitos
- No usar funciones gráficas predefinidas de Pygame
- Seguir principios de modularidad
- Cumplir estándar PEP8
- Separar interfaz y lógica
- Documentar todas las funciones
- Mantener código reutilizable y mantenible

Estructura esperada:

- Carpeta algorithms
- Carpeta ui
- Carpeta models
- Carpeta utils

Implementar:

- DDA
- Bresenham
- Circunferencia
- Curvas Bézier
- Elipses
- Polígonos

Formato esperado:

- Código modular
 - Comentarios técnicos
 - Clases reutilizables
 - Funciones pequeñas
 - Documentación clara
-

17. Control de Versiones

Versión	Fecha	Cambios
1.0	11/05/2026	Creación inicial del documento SDD

General del Proyecto

Nombre del Proyecto:

Graficador Interactivo 2D en Python

Versión:

1.0

Fecha:

11 de mayo de 2026

Autor(es):

[Pendiente por definir]

Stakeholders:

- Estudiantes del curso de gráficos computacionales
- Docente evaluador
- Desarrolladores del proyecto

Descripción General:

El sistema consiste en una aplicación gráfica interactiva desarrollada en Python utilizando la biblioteca Pygame. El software permitirá la representación visual de algoritmos clásicos de rasterización y generación de figuras geométricas en 2D mediante implementación manual de algoritmos gráficos.

Objetivo del Sistema:

Desarrollar un graficador interactivo que permita visualizar y manipular figuras geométricas usando algoritmos gráficos implementados manualmente, promoviendo el aprendizaje de rasterización y modularidad de software.

Alcance:

El sistema permitirá:

- Dibujar líneas mediante DDA y Bresenham
- Dibujar circunferencias mediante Bresenham
- Dibujar curvas Bézier cúbicas
- Dibujar triángulos, rectángulos, polígonos y elipses
- Seleccionar figuras desde una interfaz gráfica
- Configurar colores de dibujo
- Limpiar y reiniciar el lienzo
- Mantener separación entre lógica gráfica e interfaz

No se incluirán:

- Renderizado 3D
 - Persistencia en bases de datos
 - Exportación de imágenes
 - Uso de funciones gráficas predefinidas como `pygame.draw.line()`
-

2. Visión del Sistema

Problema que resuelve

Permite comprender visualmente el funcionamiento interno de algoritmos gráficos clásicos mediante implementación manual y visualización interactiva.

Usuarios objetivo

- Estudiantes de programación gráfica
- Docentes de computación gráfica
- Desarrolladores en formación

Beneficios esperados

- Comprensión práctica de algoritmos de rasterización
- Aprendizaje de modularización en Python
- Visualización inmediata de figuras geométricas
- Aplicación de buenas prácticas de programación

Contexto de uso

Aplicación educativa ejecutada en computadores con Python y Pygame instalados.

3. Requisitos Funcionales (RF)

ID	Requisito	Descripción	Prioridad	Actor
RF-01	Ventana principal	Mostrar área de dibujo y panel de control	Alta	Usuario
RF-02	Selección de figuras	Permitir seleccionar figuras geométricas	Alta	Usuario
RF-03	Algoritmo DDA	Dibujar líneas usando DDA	Alta	Usuario
RF-04	Algoritmo Bresenham línea	Dibujar líneas usando Bresenham	Alta	Usuario
RF-05	Circunferencia Bresenham	Dibujar circunferencias manualmente	Alta	Usuario
RF-06	Curvas Bézier	Dibujar curvas Bézier cúbicas	Alta	Usuario
RF-07	Formas cerradas	Dibujar triángulos, rectángulos y polígonos	Media	Usuario
RF-08	Dibujar elipses	Generar elipses en el lienzo	Media	Usuario
RF-09	Configuración de color	Seleccionar color de dibujo	Media	Usuario
RF-10	Limpiar lienzo	Reiniciar el área de dibujo	Alta	Usuario
RF-11	Renderizado inmediato	Visualizar figuras inmediatamente	Alta	Usuario
RF-12	Modularidad	Separar lógica gráfica e interfaz	Alta	Sistema
RF-13	Gestión de eventos	Capturar teclado y mouse	Alta	Sistema
RF-14	Documentación técnica	Incluir comentarios y documentación PEP8	Alta	Desarrollador

4. Requisitos No Funcionales (RNF)

ID	Tipo	Descripción
RNF-01	Rendimiento	Las figuras deben renderizarse en tiempo real
RNF-02	Modularidad	Separación entre interfaz y algoritmos
RNF-03	Mantenibilidad	Código organizado por módulos
RNF-04	Estándar	Código documentado siguiendo PEP8
RNF-05	Compatibilidad	Compatible con Python y Pygame
RNF-06	Usabilidad	Interfaz clara e intuitiva
RNF-07	Restricción técnica	No usar primitivas gráficas predefinidas de Pygame
RNF-08	Escalabilidad	Posibilidad de agregar nuevas figuras
RNF-09	Legibilidad	Funciones pequeñas y reutilizables
RNF-10	Calidad visual	Las figuras deben visualizarse correctamente

5. Casos de Uso

5.1 Diagrama de Casos de Uso

- Seleccionar figura
 - Seleccionar color
 - Dibujar línea DDA
 - Dibujar línea Bresenham
 - Dibujar circunferencia
 - Dibujar curva Bézier
 - Dibujar polígonos
 - Dibujar elipse
 - Limpiar lienzo
 - Reiniciar aplicación
-

5.2 Especificación de Casos de Uso

Nombre:

Dibujar Línea DDA

Actor:

Usuario

Descripción:

Permite dibujar una línea usando el algoritmo DDA.

Precondiciones:

- La aplicación debe estar iniciada
- El usuario debe seleccionar la herramienta DDA

Flujo principal:

1. El usuario selecciona DDA
2. El usuario define punto inicial
3. El usuario define punto final
4. El sistema calcula los píxeles
5. El sistema renderiza la línea

Flujos alternativos:

- Cancelación de selección de puntos

Postcondiciones:

- Línea renderizada en el lienzo
-

6. Modelo de Datos

6.1 Diccionario de Datos

Entidad	Atributo	Tipo	Descripción
Figura	tipo	String	Tipo de figura
Figura	color	Tuple	Color RGB
Punto	x	Integer	Coordenada horizontal
Punto	y	Integer	Coordenada vertical
CurvaBezier	puntos_control	List	Puntos de control

Interfaz herramienta_activ String Figura seleccionada
a

6.2 Diagrama Entidad-Relación (DER)

- Figura contiene múltiples puntos
 - CurvaBezier utiliza 4 puntos de control
 - Interfaz controla herramienta y color actual
-

7. Arquitectura del Sistema

7.1 Estilo Arquitectónico

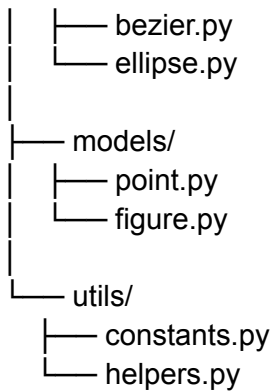
Arquitectura modular por capas.

7.2 Diagrama de Componentes

- `main.py`
 - `ui/`
 - `algorithms/`
 - `models/`
 - `utils/`
-

7.3 Diagrama de Paquetes

```
project/
|
|-- main.py
|-- ui/
|   |-- interface.py
|   |-- controls.py
|
|-- algorithms/
|   |-- dda.py
|   |-- bresenham.py
|   |-- circle.py
```



7.4 Diagrama de Clases

Clase: Point

- x
- y

Clase: Figure

- color
- draw()

Clase: DDAAlgorithm

- calculate_points()

Clase: BresenhamAlgorithm

- calculate_points()

Clase: InterfaceManager

- handle_events()
 - render()
-

8. Contratos de Interfaz (UI/UX)

Pantalla	Evento	Acción	Destino
Menú principal	Seleccionar figura	Activar herramienta	Lienzo

Lienzo	Click mouse	Capturar puntos	Algoritmo
Panel lateral	Seleccionar color	Cambiar color actual	Lienzo
Panel lateral	Limpiar	Reiniciar lienzo	Lienzo

9. Reglas de Negocio

ID	Regla	Descripción
RN-01	Implementación manual	Todos los algoritmos deben implementarse manualmente
RN-02	Restricción gráfica	No usar <code>pygame.draw.line()</code>
RN-03	Curva Bézier	Debe utilizar 4 puntos de control
RN-04	Modularidad	Separar GUI y lógica
RN-05	Color activo	Todas las figuras usan el color seleccionado
RN-06	Interacción	La figura debe mostrarse inmediatamente
RN-07	Documentación	Todas las funciones deben documentarse

10. Especificación de Algoritmos

Nombre del algoritmo:

DDA

Entrada:

- Punto inicial (`x1, y1`)
- Punto final (`x2, y2`)

Salida:

Lista de píxeles de la línea.

Descripción:

Calcula incrementos en X y Y para interpolar los puntos de una línea.

Inicio

1. Calcular dx y dy
2. Determinar pasos
3. Calcular incrementos
4. Iterar sobre puntos
5. Dibujar píxeles

Fin

11. API / Servicios

Endpoint	Método	Descripción
No aplica	No aplica	Aplicación local sin servicios externos

12. Trazabilidad

RF	Caso de Uso	Módulo	Prueba
RF-03	Dibujar línea DDA	dda.py	PT-01
RF-04	Línea Bresenham	bresenham.py	PT-02
RF-05	Circunferencia	circle.py	PT-03
RF-06	Curva Bézier	bezier.py	PT-04
RF-10	Limpiar lienzo	interface.py	PT-05

13. Plan de Pruebas

ID	Descripción	Entrada	Resultado Esperado
PT-01	Línea DDA	Dos puntos	Línea correcta
PT-02	Bresenham	Dos puntos	Línea rasterizada
PT-03	Circunferencia	Centro y radio	Circunferencia visible

PT-04	Bézier	4 puntos	Curva renderizada
PT-05	Limpiar lienzo	Botón limpiar	Lienzo vacío

14. Criterios de Aceptación

El sistema debe permitir:

- Seleccionar figuras desde la interfaz
- Dibujar figuras en tiempo real
- Cambiar colores
- Limpiar el lienzo
- Visualizar algoritmos manuales

El sistema debe validar:

- Entrada correcta de puntos
 - Selección válida de herramientas
 - Separación modular del código
 - Cumplimiento de documentación PEP8
-

15. Gestión del Proyecto

Metodología:

Desarrollo incremental por módulos.

Herramientas:

- Python
- Pygame
- VS Code
- Git

Cronograma:

- Diseño de interfaz
- Implementación de algoritmos

- Integración GUI
 - Pruebas
 - Documentación
 - Video demostrativo
-

16. Integración con Agentes de IA

16.1 Roles

Agente	Responsabilidad
Agente Arquitecto	Diseñar arquitectura modular
Agente Desarrollador	Implementar algoritmos
Agente QA	Validar funcionamiento
Agente Documentador	Generar documentación técnica

16.2 Prompt Base

Actúa como un desarrollador experto en Python y Pygame.

Usa esta especificación SDD para implementar un graficador interactivo 2D.

Restricciones:

- No inventar requisitos
- No usar funciones gráficas predefinidas de Pygame
- Seguir principios de modularidad
- Cumplir estándar PEP8
- Separar interfaz y lógica
- Documentar todas las funciones
- Mantener código reutilizable y mantenible

Estructura esperada:

- Carpeta algorithms
- Carpeta ui
- Carpeta models
- Carpeta utils

Implementar:

- DDA
- Bresenham
- Circunferencia
- Curvas Bézier
- Elipses
- Polígonos

Formato esperado:

- Código modular
 - Comentarios técnicos
 - Clases reutilizables
 - Funciones pequeñas
 - Documentación clara
-

17. Control de Versiones

Versión	Fecha	Cambios
1.0	11/05/2026	Creación inicial del documento SDD